Project Assignment: Nmap Scan Parser & Dashboard System

Domain: Network Security / Backend Development / Database Management
Technology Stack: Python, Flask, MySQL, Nmap, Bootstrap

1. Project Objective

Build a web-based Network Asset Discovery and Nmap Scan Management System. The application should execute selected Nmap commands, parse XML scan results, store structured data in MySQL, and display the results in a searchable dashboard.

2. Functional Requirements

- Accept target IP address, IP range, or hostname from user input.
- Execute predefined Nmap commands from the application.
- Generate XML output using Nmap (-oX option).
- Parse XML scan results using Python.
- Store parsed data into MySQL database.
- Display scan results in dashboard format.
- Maintain scan history and timestamps.
- Allow filtering/searching based on host, port, service, and status.

3. Mandatory Nmap Commands to Support

No.	Purpose	Command
1	Ping Scan	nmap -sn <target>
2	Fast Scan	nmap -F <target>
3	Service Version Detection	nmap -sV <target>
4	OS Detection	nmap -O <target>
5	TCP SYN Scan	nmap -sS <target>
6	TCP Connect Scan	nmap -sT <target>
7	UDP Scan	nmap -sU <target>
8	Aggressive Scan	nmap -A <target>
9	Scan Specific Ports	nmap -p 22,80,443 <target>
10	Top Ports Scan	nmap --top-ports 100 <target>
11	Full Port Scan	nmap -p- <target>
12	Detect HTTP Title	nmap --script http-title <target>
13	Vulnerability Script Scan	nmap --script vuln <target>
14	Network Range Scan	nmap 192.168.1.0/24
15	Save XML Output	nmap -sV <target> -oX scan.xml

4. XML Parsing Requirements

- Parse XML using ElementTree, lxml, or BeautifulSoup XML parser.
- Extract Host IP Address.
- Extract Hostname.
- Extract Host Status (up/down).
- Extract Port Number and Protocol.
- Extract Port State (open/closed/filtered).
- Extract Service Name and Version Information.
- Extract Operating System Details if available.
- Store raw XML scan file for reference.

5. Suggested Database Schema

Table: scans
- id (Primary Key)
- target
- scan_type
- command_used
- scan_time
- raw_xml_path

Table: hosts
- id (Primary Key)
- scan_id (Foreign Key)
- ip_address
- hostname
- host_status
- os_details

Table: ports
- id (Primary Key)
- host_id (Foreign Key)
- port_number
- protocol
- port_state
- service_name
- service_version

6. Dashboard Requirements

- Dashboard homepage with total scans count.
- Show total hosts discovered.
- Show total open ports.
- Display latest scans.
- Search/filter by IP address.
- Search/filter by port number.
- Search/filter by service name.
- Display detailed host information page.
- Display open ports for each host.
- Show scan execution time and command used.

7. Optional Advanced Features

- Generate PDF scan reports.
- Add charts using Chart.js.
- Detect risky/common vulnerable ports.
- Compare old and new scans.
- Export data to CSV.
- Dockerize the application.
- Schedule periodic scans.
- Add login authentication.

8. Development Guidance

- Use Flask for backend API and routing.
- Use Bootstrap for frontend UI.
- Use subprocess module carefully to execute Nmap commands.
- Always sanitize user input before executing commands.
- Store XML files separately in a scans/ folder.
- Test using localhost or private network IP ranges only.
- Do not scan public internet targets.

9. Suggested Project Timeline

Timeline	**Task**
Day 1-2	Understand Nmap and XML output structure
Day 3-4	Implement Flask project structure
Day 5-6	Execute Nmap commands from Python
Day 7-8	Implement XML parsing
Day 9-10	Design MySQL schema and integrate database
Day 11-12	Build dashboard UI
Day 13	Testing and debugging
Day 14	Documentation and screenshots
Day 15	Final Submission

10. Final Deliverables

- Source code
- Database schema SQL file
- Sample XML scan files
- Working application demo
- Project report/documentation
- Screenshots of dashboard
- GitHub repository (optional)